Library WordExcel

La clase WordExcel ha sido concebida como una biblioteca de funciones (métodos) que dan apoyo al programador de C# y que ahorrarían gran cantidad de líneas de programación. Por otra parte, disminuye en gran medida la pérdida de tiempo producida por el **"cómo se hace"** en tiempo de programación. Este documento esta básicamente orientado al programador neófito de C# y por tal razón se trata de ser lo más explícito posible en el documento.

Dicha clase viene representada por la biblioteca dinámica WE.DLL adjunta a este proyecto. Para poder trabajar con los métodos de la clase WE se deben seguir los siguientes pasos:

Paso Principal y Obligatorio:

Para trabajar con Word o con Excel se debe agregar el paquete NuGet ClosedXML (que incluye el SDK oficial de Microsoft DocumentFormat.OpenXml), tal y como se describe más adelante al inicio de la sección en forma gráfica:

==> **MÉTODOS PARA GENERACIÓN DE DOCUMENTOS WORD**

Esto se hace en Visual Studio 2022 en:

Herramientas->Administrador de paquetes Nuget->Consola de administración de paquetes

y cuando salga el prompt **PM>** se debe teclear:

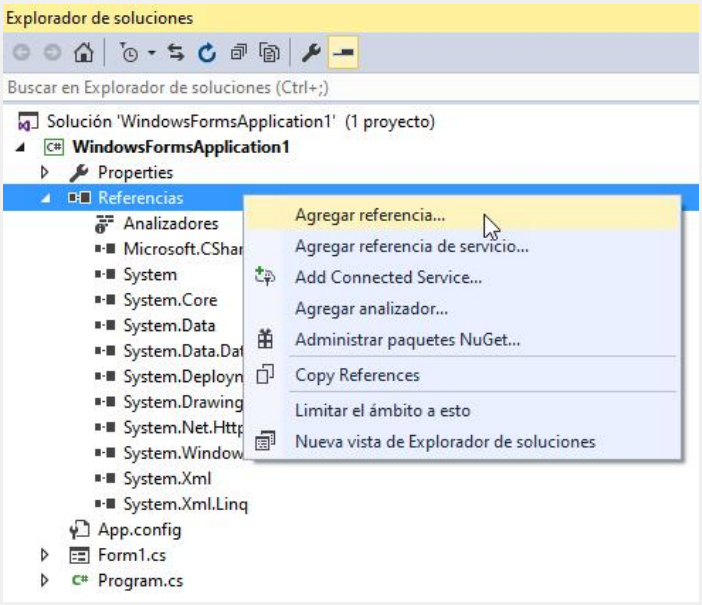
Install-Package ClosedXML -Version 0.102.2 (entonces esperar a que termine dicho proceso)

Este mismo paquete sirve para Word y para Excel, por lo que no hace falta instalar ningún otro.

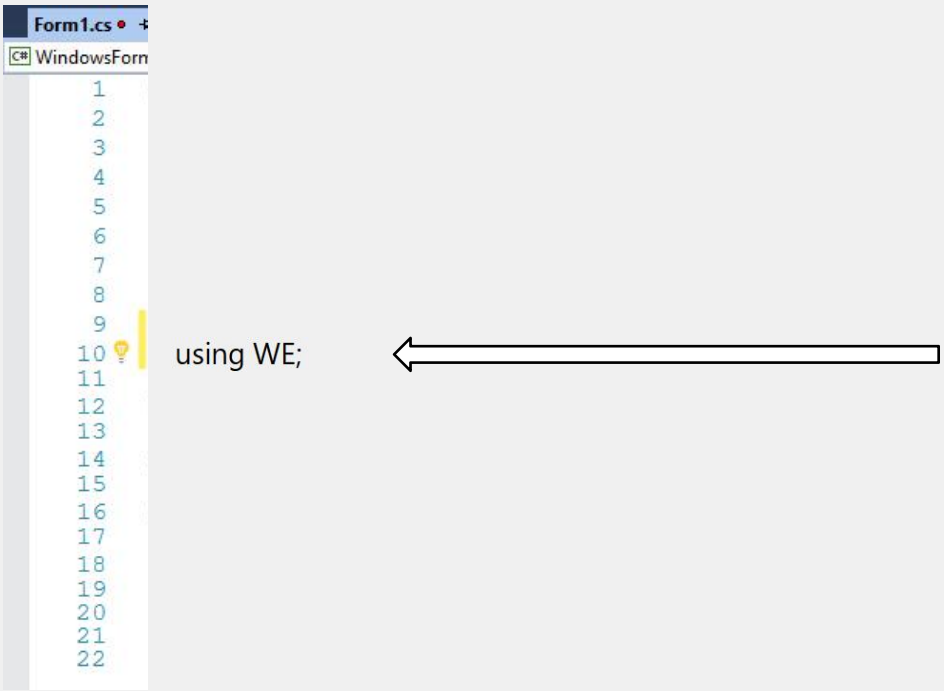
1) Después comenzamos agregando una referencia a dicha DLL (WE.DLL) que queda después de la instalación, en el escritorio de la pantalla. Es conveniente que inmediatamente después de la instalación, usted copie dicha DLL a una carpeta personal a fin de agregar siempre las referencias desde ese lugar personalizado, pues el escritorio es inseguro y en cualquier momento se puede borrar por equivocación a dicha DLL y esto implicaría la necesidad de volver a instalar la aplicación. Lo mismo sugerimos con los manuales de usuario (En español e inglés) y los ficheros asociados al programa de ejemplo, que se deben colocar en la raíz del disco D:\, para que corra correctamente.

Nota importante:

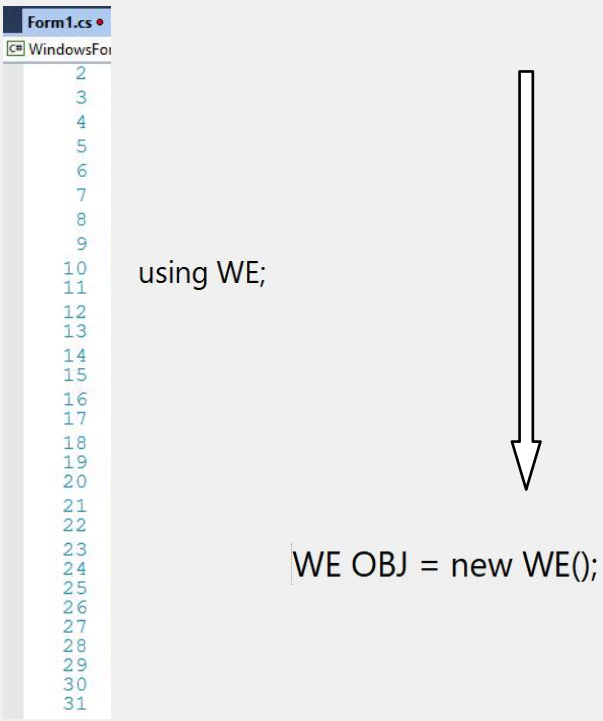
Si usted desea también agregar la biblioteca multifuncional LN.DLL, entonces no es necesario agregar ningún Package del tipo Nuget, simplemente adiciónela como una referencia normal. Al finalizar la instalación de LN.DLL queda también en el escritorio y por tal razón le recomendamos lo mismo que para WE.DLL, cópiela en una carpeta personalizada suya.



2) Seguidamente agregamos el espacio de nombres WE mediante el **using** correspondiente dentro del código de programa en la forma tradicional:



3) A partir de este momento ya se puede instanciar el objeto con el nombre deseado (en nuestro caso, denominado OBJ):



WE OBJ = new WE();

4) Ahora al escribir **OBJ** seguido de un punto rápidamente le aparecerá la lista completa de todos los métodos de dicha clase, y bastará con seleccionar el método deseado.

La clase WE está concebida fundamentalmente para aplicaciones de tipo **Winforms (.exe)**, pero en el documento se especifica claramente cuales métodos pueden ser utilizados en aplicaciones **WEB** y cuales en aplicaciones **Winforms**.

METODOS DE LA CLASE WE

MÉTODOS PARA GENERACIÓN DE DOCUMENTOS WORD 5
Métodos de Word 9
Método de Apertura y Creación de un Documento Word 9
Método de Creación de un Heading de tipo Título1, Título2, etc. 10
Método de Cierre del Documento Word 11
Método de Inserción de una Imagen en el Documento Word 12
Método de Creación de una Tabla 13
Método de Creación del Header Columnar 14
Método de Creación del Header con letra Itálica 15
Método de Escritura del Contenido de una Celda 15
Métodos para Encabezamiento y Pie De Página 16
Método de Creación del Pié de Página 17
Método de Adición de un Párrafo en el Documento 18
Programa de Ejemplo de Creación de un Documento Word 19
Documento .docx Generado 21
Métodos de Excel 22
Metodo ExcelCreateDocExcel() de Creación de un Documento EXCEL con su Sheet Inicial 22
METODO ExcelOpenFile(string PathFile) 23
(Apertura de un Documento Excel) 23
METODO ExcelSaveFile() DE SALVA DEL DOCUMENTO EXCEL 23
METODO ExcelFormatTitles() DE ESCRITURA DE UN TITULO COLUMNAR (Row = 1, Column = n) 24
METODO ExcelFormatCell() DE DAR FORMATO A UNA CELDA 25
METODO ExcelWriteCellRC() DE ESCRITURA DE UNA CELDA DEL 26
DOCUMENTO EXCEL 26
METODO ExcelWriteCell() DE ESCRITURA DE UNA CELDA DEL DOCUMENTO EXCEL 26
METODO ExcelGetCellRC() DE ESCRITURA DE UNA CELDA CON NOMBRE EN string 27
METODO ""ExcelGetCell" DE LECTURA DE UNA CELDA DEL DOCUMENTO EXCEL 27
METODO ExcelGetCellDataTypeRC() DE LECTURA DEL TIPO DE DATOS DE UNA CELDA (r, c) 28
METODO ExcelGetCellDataType() DE LECTURA DEL TIPO DE DATO DE UNA CELDA EN FORMATO "string" 28
METODO ExcelFormula() PARA DESCRIBIR UNA FORMULA EN EXCEL 29
Programa de ejemplo de generación de un documento EXCEL 30
METODOS DE DataGridView 34
Método DGVColumnSumarize() 34
Método DGVColumnSumarizeSelectedRows() 34
Método DGVColumnAverage() 35
Método DGVColumnAverageSelectedRows() 35
Método DGVGetCell() 36
Método DGVPutCell() 36

MÉTODOS PARA GENERACIÓN DE DOCUMENTOS WORD

A continuación se explica en detalle un conjunto de métodos desarrollados para facilitar al programador de C# el trabajo de generación de un documento Word (.docx, .doc).

Para la realización de los mismos hemos escogido el SDK oficial de Microsoft para documentos Office, el paquete NuGet DocumentFormat.OpenXml (versión 2.16.0), que se distribuye con licencia MIT y permite su uso libre, también en aplicaciones comerciales. Hasta octubre de 2026 estos métodos usaban el paquete Docx de Xceed, cuya licencia solo permite el uso no comercial, y por eso se sustituyó.

El SDK Open XML ha sido desarrollado por Microsoft; su documentación y su código están en:

https://learn.microsoft.com/es-es/office/open-xml/open-xml-sdk (Documentación en español)
https://github.com/dotnet/Open-XML-SDK (Código fuente y licencia MIT)

La ventaja que he observado en el mismo, es que con este paquete se puede realizar este tipo de trabajo sin necesidad de tener instalado el Microsoft Office en su PC, en mi caso personal donde uso el Ashampoo Write, que es un equivalente al Word de Microsoft, funcionó perfectamente y es de suponer que funcione de igual forma en todos aquellos productos que sirven para la creación de documentos .docx compatibles con MSWord.

Si el usuario tiene instalado el Microsoft Office, entonces puede acceder al Word según se muestra a continuación:

```
//=========================================================
using System;
using System.IO;
//Interop no se localiza si no tenemos el Office de Microsoft instalado
using Microsoft.Office.Interop.Word;
class Program
{
static void Main()
{
// Crear una instancia de la aplicación Word
Application wordApp = new Application(); // Con la clase Application se instancia la nueva app
// * * * Crear un nuevo programa
//=========================================================
```

En mi criterio personal, el SDK de Microsoft es muy completo pero también muy minucioso: para escribir un simple párrafo con formato hay que construir a mano varios elementos del documento, y al inicio se resiste un poco para romper la inercia y comenzar a programar con fluidez y en forma sencilla.

Por lo dicho anteriormente he desarrollado un conjunto de métodos con una lógica clara y sencilla, que funcionan perfectamente y que ayuda a ahorrar tiempo de programación.

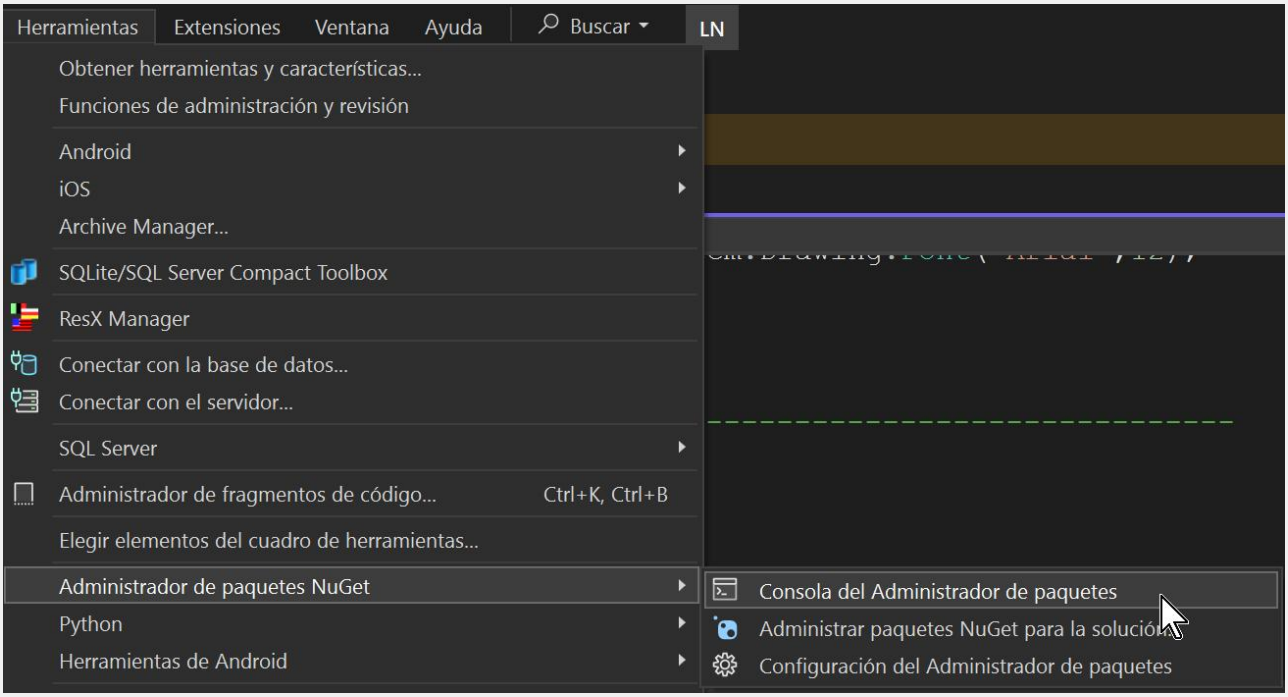
Conjuntamente con este documento les adjunto un ejemplo que usa todos los métodos desarrollados.

La misma fue programada con:

Microsoft Visual Studio Community 2022 (64 bits)
Versión 17.9

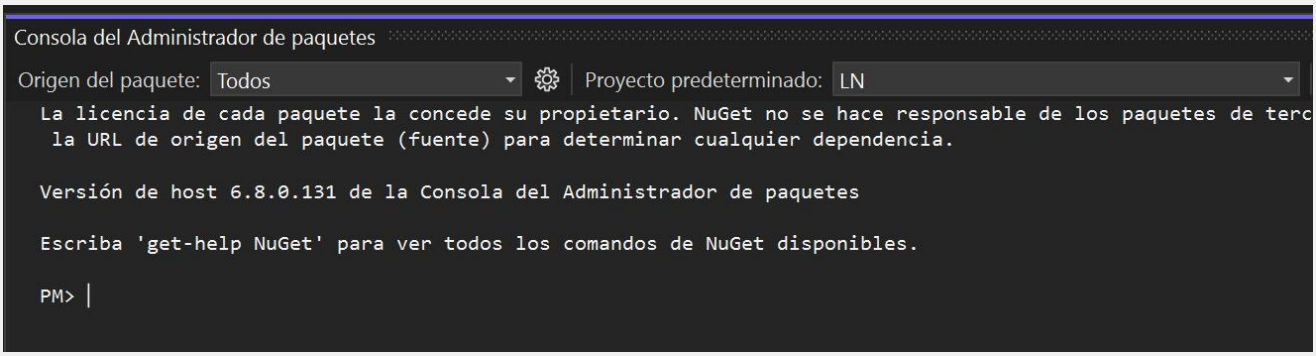
con el Package NuGet 6.8.0

Lo primero que se debe hacer para tener acceso a estos métodos, es instalar el paquete ClosedXML (versión 0.102.2), que trae incluido el SDK DocumentFormat.OpenXml, en la consola de trabajo de Nuget dando click en Herramientas del menú de Visual Studio en la forma siguiente:



y rápidamente aparecerá dicha consola de trabajo en la parte inferior de la ventana del Visual Studio según se muestra a continuación:

WordExcel;



Y aquí se escribe:

```
PM> Install-Package ClosedXML -Version 0.102.2
```

La primera vez aparecerán varias líneas que le dirán cómo ha sido el proceso de instalación; si el paquete ya estaba instalado en el proyecto, se emite el mensaje correspondiente.

Es conveniente aclarar que el paquete ClosedXML se instala dentro de la solución que dará uso del mismo, por lo que cada vez que se quiera usar dicha librería en otra solución, pues debemos repetir estos pasos anteriores dentro de dicha solución, a menos que se disponga de una biblioteca .DLL que contenga a estos Packages ya incorporados y un conjunto de métodos creados a tal efecto, como es nuestro caso, pues en la biblioteca WE, se encuentran varios métodos que permiten trabajar fácilmente con Word y con Excel, entre otras muchas cosas.

A continuación se declara la using asociada, de la forma siguiente:

```
//=================================================================
using WE; // Métodos de Word y Excel de la biblioteca WE
//=================================================================
```

Y ahora ya estamos listos para usar los métodos que describiremos a continuación pertenecientes a la biblioteca **WE**.

Lo primero que se debe hacer es instanciar un objeto OBJ en la zona de variables globales de la clase principal del programa que va a utilizar a esta biblioteca, por ejemplo:

```
    public partial class Form1 : Form
        {
        //-------------------------------------------------
        // Area de definición de variables globales a la clase
        //-------------------------------------------------
        LN.LN OBJ1 = new LN.LN();
        WE.WE OBJ  = new WE.WE();
        //WE.WordDoc   Documento; // Clase WordDoc usada
        //                                  // para definir a
        //                                  // Documento
        //-------------------------------------------------
        public Form1()
               {
               ...
```

Nota Importante:
Personalmente siempre he pensado que no hay nada más ilustrativo para aprender que mostrar varios ejemplos. Por tal razón, en ambos casos, acompañamos a esta documentación con algunos ejemplos de uso que seguramente los ayudará en el ahorro de tiempo de programación, cuestión esta que es bien importante y espero que estén de acuerdo conmigo. De ahora en adelante usaremos al objeto instanciado **OBJ** para calificar a los distintos métodos de la clase **WE** (**W**ord & **E**xcel) cuando se usen.

Métodos de Word

Método de Apertura y Creación de un Documento Word

WE.WordDoc OpenWordDoc(string PathDoc)

Descripción:

OpenWordDoc es el primer método a ejecutar en la creación de un documento Word y realiza la apertura del mismo.

El parámetro **PathDoc** de tipo **string** define el **Path** del documento que se va a crear y el nombre del mismo.

Al finalizar la ejecución se retorna el objeto asociado "**ObjDoc**", que va a ser referenciado a partir de este momento por los restantes métodos asociados.

"**ObjDoc**" es un objeto de la clase "**WE.WordDoc**".

Este objeto retornado se guarda también en un buffer que es una variable global **"Documento"** de WE para futuros usos.

Ejemplo:
En este ejemplo se crea un nuevo documento "**D:\AAA.docx** y al objeto retornado del tipo WE.WordDoc se le ha llamado **ObjDoc** el cual se copia también

```
WE.WordDoc ObjDoc = OBJ.OpenWordDoc(@"D:\AAA.docx");
WE.WordDoc DocBuffer = ObjDoc;
```

Nota Importante:
La secuencia anterior podía haber sido escrita también en la forma siguiente:

```
var ObjDoc = OBJ.OpenWordDoc(@"D:\AAA.docx");
```

usando el tipo de datos genérico **var.**

Parámetros de Entrada:
string PathDoc (define el Path del documento a crear)

Parámetros de salida:
No tiene.

Retorno:
WE.WordDoc ObjDoc

Método de Creación de un Heading de tipo Título1, Título2, etc.

```
public void AddTitleType(WE.WordDoc DocName,
      string                                TitleTXT,
      WE.HeadingType    TitleType,
      string                                TitleFontType,
      int                                   TitleFontSize,
      bool                                  TitleBoldType,
      bool                                  TitleItalicType,
      System.Drawing.Color                  TitleColor,
      WE.Alignment              TitleAligment)
```

Descripción:

El método "AddTitleType" de tipo "void" es utilizado para crear un párrafo de tipo Título1, 2, etc.

Contiene 9 parámetros:
"**DocName**" de la clase "WE.WordDoc" define el nombre del documento Word retornado en OpenWordDoc.
"**TitleTXT**" de tipo string, define el texto a mostrar.
"**TitleType**" es un objeto de la clase WE.HeadingType que define si el texto dado estará en Titulo1 (Heading1), Título2 (Heading2), etc.
"**TitleFontType**" de tipo string, define el tipo de letra, por ejemplo "Arial".
"**TitleFontSize**" de tipo int, define el tamaño de la letra, por ejemplo 14.
"**TitleBoldType**" de tipo bool, si es true significa que se desea letra "bold".
"**TitleItalicType**" de tipo bool, si es true significa que se desea letra "Itálica".
"**System.Drawing.Color TitleColor**", define el color de la letra, por ejemplo:
System.Drawing.Color.**Red**

define color rojo, si escogemos System.Drawing.Color.**DarkRed** entonces es rojo oscuro.
"WE.Alignment TitleAligment", define la alineación del pié de página, por ejemplo, si escribimos "**WE.Alignment.center**", entonces el texto aparecerá centrado.

Parámetros de entrada:

```
WE.WordDoc              DocName,
string                  TitleTXT,
WE.HeadingType      TitleType,
string                  TitleFontType,
int                     TitleFontSize,
bool                    TitleBoldType,
bool                    TitleItalicType,
System.Drawing.Color        TitleColor,
WE.Alignment        TitleAligment)
```

Parámetros de Salida:
No tiene

Retorno:
No tiene

Ejemplo:

```
ObjDoc.AddTitleType(ObjDoc,
                    "PARRAFO1",
                     WE.HeadingType.Heading1,
                     "Arial", 16, false, true, System.Drawing.Color.Black,
                WE.Alignment.both);
```

en este caso se adiciona un párrafo a partir de la línea en curso con el texto:
PARRAFO1, de tipo **Heading1** (Titulo1), con letra Arial de tamaño 16, sin letra negrita (Bold) de tipo de letra Itálica, con color de la fuente en negro y alineado en ambos lados.

Método de Cierre del Documento Word

Descripción:

Es el último método a ejecutar en la creación de un documento Word y es el que crea y guarda a dicho documento en el disco.
Después de ejecutado el "CloseWordDoc" no se permite el uso de los restantes métodos sobre el documento en curso, a menos que se vuelva a crear otro documento nuevo.
El parámetro DocOpen de tipo "**WE.WordDoc**" define el nombre que fue retornado por el método "**OpenWordDoc**".

Nota: Internamente lo que se ejecuta es el método "Save" de WE.WordDoc, que guarda el documento.

Formato:

void **CloseWordDoc**(WE.WordDoc DocOpen)

Parámetro de Entrada:

WE.WordDoc **DocOpen**
donde DocOpen es el nombre que fue retornado por **OpenWordDoc**

Parámetro de Salida:
No tiene

Ejemplo:
OBJ.CloseWordDoc(**ObjDoc**);

Método de Inserción de una Imagen en el Documento Word

void **InsertImageWordDoc**(WE.WordDoc DocOpen,
string Imagen)

Descripción:
El método "InsertImageWordDoc", esta destinado a la inserción de una imagen (.jpg, .bmp, etc.), en la línea en curso, dentro del proceso de creación del documento y requiere de dos parámetros obligatorios:
1) "DocOpen" del tipo "WE.WordDoc" que es el nombre que fué retornado por "OpenWordDoc".
2) El segundo operando se da el path de la imagen de tipo "string".
Es conveniente aclarar que al finalizar se escribe una línea de string "***" para evitar conflictos posteriores en Ashampoo Office Write, que es un equivalente al Word de Microsoft pero más barato. Esto ocurre cuando se crea una tabla inmediatamente después de otra tabla, lo cual provoca que el párrafo siguiente no se visualiza en dicha aplicación, también el WordPad de Windows 10 detecta el problema pero advierte que esto pudiera suceder en apps equivalentes al Word de Microsoft.
Este bug, es de suponer que sea arreglado en algún Service Pack de la app.

Parámetros de entrada:
DocOpen
Imagen

Parámetros de salida:
No tiene
Ejemplo de uso:

```
OBJ.InsertImageWordDoc(ObjDoc, @"D:\FotoPaisaje.jpg");
```

Método de Creación de una Tabla

```
WE.WordTable AppendTable(WE.WordDoc   DocName,
                         int      TableRows,
                         int      TableColumns)
```

Descripción:

El método **"AppendTable"** realiza la creación de una tabla vacia en Word en la línea en curso, de modo tal que solo tiene en cuenta el número de filas y columnas de la misma.
Este método tiene tres parámetros de entrada:
1) DocName que es el nombre que fué retornado por "**OpenWordDoc"** del tipo **WE.WordDoc**
2) TableRows es un parámetro numérico de tipo "int" que define la cantidad de filas de la tabla.
3) TableColumns es un parámetro numérico de tipo "int" que define la cantidad de columnas de la tabla.

Al término de la ejecución se retorna el nombre interno de la tabla, el cual es retornado con el tipo de la clase **"WE.WordTable"** y que es el que debe ser usado posteriormente en todos los métodos asociados al trabajo con tablas.

Parámetros de entrada:
DocName
TableRows
TableColumns

Parámetros de salida:
No tiene

Retorno: El nombre de la tabla bajo el formato:
WE.WordTable TABLA

Método de Creación del Header Columnar

```
void PutHeaderName(WE.WordTable  TableName,
                   int                    TableCell,
                   string                 ColumnHeader,
                   string                 ColumnHeaderFontType,
                   int                    ColumnHeaderFontSize,
                   System.Drawing.Color   ColumnHeaderColor)
```

Descripción:

El método **"PutHeaderName"** de tipo "void" es el que se ocupa de escribir uno a uno cada título de encabezamiento en cada celda (Columna) de la fila 0 (Fila de encabezado).

Contiene 6 parámetros:

"TableName" del tipo (WE.WordTable)
"TableCell" del tipo (int)
"ColumnHeader" del tipo (string)
"ColumnHeaderFontType" del tipo (string)
"ColumnHeaderFontSize" del tipo (int)
"ColumnHeaderColor" del tipo (System.Drawing.Color)

Las celdas de las filas se escriben alineadas al centro.
En **"TableName"** aparece el nombre que retornó el método "AppendTable" como nombre de tabla
En **"TableCell"** se da el número de columna del Header Columnar (int).
En **"ColumnHeader"** se define el título de la columna (string).
En **"ColumnHeaderFontType"** define la letra que se va a usar para el ColumnHeader (string).
En **"ColumnHeaderFontSize"** se defne el tamaño de la letra (int).
En **"ColumnHeaderColor"** define el color de la letra (System.Drawing.Color).
El número de las columnas (Cells) son relativos a 0 al igual que el de las filas (Rows).

Parámetros de entrada:

"TableName" del tipo (WE.WordTable)
"TableCell" del tipo (int)
"ColumnHeader" del tipo (string)
"ColumnHeaderFontType" del tipo (string)
"ColumnHeaderFontSize" del tipo (int)
"ColumnHeaderColor" del tipo (System.Drawing.Color)

Parámetros de salida:

No tiene.

Retorno:
No tiene

Método de Creación del Header con letra Itálica

Este método es similar al anterior con la pequeña diferencia de que los títulos de las columnas se crean con letra ***Itálica***.

Método de Escritura del Contenido de una Celda

```
void PutInfoCell(WE.WordTable      TableName,
                 string                    TableRowInfo,
                 int                       TableRow,
                 int      T                ableCell)
```

Descripción:

El método "PutInfoCell" de tipo "void" es el que se ocupa de escribir uno a uno el contenido de cada celda en una fila dada.

Contiene 4 parámetros:

"TableName" del tipo (WE.WordTable)
"TableRowInfo" del tipo (string)
"TableRow" del tipo (int)
"TableCell" del tipo (int)

Las celdas de las filas se escriben alineadas a la izquierda.
En **"TableName"** aparece el nombre que se le dió a la tabla mediante la clase "WE.WordTable"
En **"TableRowInfo"** se especifica el contenido que se desea escribir en la celda en formato string.
En **"TableRow"** se da el número de la fila que contiene la celda a escribir.
En **"TableCell"** se da el número de la celda (columna) donde se escribirá el contenido deseado.

El número de las columnas (Cells) son relativos a 0 al igual que el de las filas (Rows).

Parámetros de entrada:

"TableName" del tipo (WE.WordTable)
"TableRowInfo" del tipo (string)
"TableRow" del tipo (int)
"TableCell" del tipo (int)

Parámetros de salida:
No tiene.

Retorno:
No tiene.

Métodos para Encabezamiento y Pie De Página

```
void CreateDocHeader(WE.WordDoc        DocName,
                     string                    HeaderTxt,
                     string                    HeaderFontType,
                     int                       HeaderFontSize,
                     bool                      HeaderBoldType,
                     bool                      HeaderItalicType,
                     System.Drawing.Color      HeaderColor,
                     WE.Alignment        HeaderAligment)
```

Descripción:

El método "CreateDocHeader" de tipo "void" solo es utilizado para crear encabezamiento de páginas caso de que se desee.

Contiene 8 parámetros:
"DocName" de la clase "WE.WordDoc" define el nombre del documento Word.
"HeaderTxt" de tipo string, define el texto de encabezamiento de las páginas.
"HeaderFontType" de tipo string, define el tipo de letra a usar, por ejemplo "Arial".
"HeaderFontSize" de tipo int, define el tamaño de la letra, por ejemplo 14.
"HeaderBoldType" de tipo bool, si es true significa que se desea letra "bold".
"HeaderItalicType" de tipo bool, si es true significa que se desea letra "Itálica".
"System.Drawing.Color HeaderColor", define el color de la letra, por ejemplo:
System.Drawing.Color.Red define color rojo, si escogemos:
System.Drawing.Color.DarkRed entonces es rojo oscuro.
"WE.Alignment HeaderAligment", define la alineación del texto de encabezamiento, por ejemplo, si escribimos "WE.Alignment.center", entonces el texto aparecerá centrado.
Nota: Usualmente los colores que se escojen con este fin son atenuados (no dark).

Parámetros de entrada:

```
WE.WordDoc          DocName,
string                          HeaderTxt,
string                          HeaderFontType,
int                             HeaderFontSize,
bool                            HeaderBoldType,
bool                            HeaderItalicType
System.Drawing.Color        HeaderColor,
WE.Alignment    HeaderAligment)
```

Parámetros de salida:

No tiene.

Retorno:

No tiene.

Método de Creación del Pié de Página

```
void CreateDocFooter(WE.WordDoc         DocName,
                string                          FooterTXT,
                string                          FooterFontType,
                int                             FooterFontSize,
                bool                            FooterBoldType,
                bool                            FooterItalicType,
                System.Drawing.Color            FooterColor,
                WE.Alignment        FooterAligment)
```

Descripción:

El método "CreateDocFooters" de tipo "void" solo es utilizado para crear el pié de página caso de que se desee.
Contiene 8 parámetros:
"**DocName**" de la clase "WE.WordDoc" define el nombre del documento Word.
"FooterTXT" de tipo string, define el texto del pié de página; a continuación de este texto se escribe automáticamente el número de cada página.
"**FooterFontType**" de tipo string, define el tipo de letra, por ejemplo "Arial".
"**HeaderFontSize**" de tipo int, define el tamaño de la letra, por ejemplo 14.
"**HeaderBoldType**" de tipo bool, si es true significa que se desea letra "bold".
"**HeaderItalicType**" de tipo bool, si es true significa que se desea letra "Itálica".
"**System.Drawing.Color FooterColor**", define el color de la letra, por ejemplo:
System.Drawing.Color.Red define color rojo, y si escogemos:
System.Drawing.Color.DarkRed entonces es rojo oscuro.
"**WE.Alignment FooterAligment**", define la alineación del pié de página, por ejemplo, si escribimos "WE.Alignment.center", entonces el texto aparecerá centrado.
Nota: Usualmente los colores que se escojen con este fin son atenuados (no dark).

Parámetros de entrada:

```
WE.WordDoc              DocName,
string                  FooterTXT,
string                  FooterFontType,
int                     FooterFontSize,
bool                    FooterBoldType,
bool                    FooterItalicType,
System.Drawing.Color    FooterColor,
WE.Alignment    FooterAligment)
```

Parámetros de salida:

No tiene.

Retorno:

No tiene.

Método de Adición de un Párrafo en el Documento

```
void AddParagraph(WE.WordDoc            DocName,
        string                          ParagraphTXT,
        string                          ParagraphFontType,
        int                             ParagraphFontSize,
        bool                            ParagraphBoldType,
        bool                            ParagraphItalicType,
        System.Drawing.Color            ParagraphColor,
        WE.Alignment    ParagraphAligment)
```

Descripción:

El método "AddParagraph" de tipo "void" es utilizado para adicionar un nuevo párrafo al documento.
Contiene 8 parámetros:
"DocName" de la clase "WE.WordDoc" define el nombre retornado en la
creación del documento.
"ParagraphTXT" de tipo string, define el texto del párrafo a escribir.
"ParagraphFontType" de tipo string, define el tipo de letra a usar en el párrafo,
por ejemplo "Arial".
"ParagraphFontSize" de tipo int, define el tamaño de la letra, por ejemplo 12.
"ParagraphBoldType" de tipo bool, si es true significa que se desea letra "bold".
"ParagraphItalicType" de tipo bool, si es true significa que se desea letra
"Itálica".
"System.Drawing.Color ParagraphColor", define el color de la letra, por ejemplo
System.Drawing.Color.Red define color rojo, y si escogemos:
System.Drawing.Color.DarkRed entonces es rojo oscuro.
"WE.Alignment ParagraphAligment", define la alineación del párrafo, por ejemplo, si escribimos
"WE.Alignment.center", entonces el texto aparecerá centrado.

Parámetros de entrada:

WE.WordDoc DocName,
string ParagraphTXT,
string ParagraphFontType,
int ParagraphFontSize,
bool ParagraphBoldType,
bool ParagraphItalicType,
System.Drawing.Color ParagraphColor,
WE.Alignment ParagraphAligment)

Parámetros de salida:
No tiene.

Retorno:
No tiene.

Programa de Ejemplo de Creación de un Documento Word

La mejor forma de entender un conjunto de métodos nuevos relativos a un tema específico es viendo un ejemplo de uso del código y posteriormente ver el resultado.

A continuación se muestra un sencillo programa que utiliza todos los métodos anteriores. Este sencillo programa es del tipo WinForms con un solo botón que cuando se da click, ejecuta al programa.

```
using LN;
using WE;
namespace PruebaWord
    {
    public partial class Form1 : Form
        {
        //-----------------------------------------------------
        // Area de definición de variables globales a la clase
        //-----------------------------------------------------
        LN.LN OBJ = new LN.LN();
        WE.WE WEOBJ  = new WE.WE();
        //WE.WordDoc   Documento; // Clase WordDoc usada
        //                                 // para definir a
        //                                 // Documento
        //-----------------------------------------------------
        public Form1()
            {
            InitializeComponent();
            }

        private void button1_Click(object sender, EventArgs e)
            {
            var ObjDoc = WEOBJ.OpenWordDoc(@"D:\AAA.docx");
            // Documento = ObjDoc; // Esto es si se necesita guardar a ObjDoc en un buffer Documento
            // para alguna acción posterior. En nuestro caso no es necesario
            //=========================<CREACION DE LOS HEADERS>======================
            WEOBJ.CreateDocHeader(ObjDoc, "Encabezado del Documento", "Arial", 14, true, true,
                                  System.Drawing.Color.Gray,
                                  WE.Alignment.right);
```

```
            //=========================<CREACION DE LOS HEADERS>======================
            WEOBJ.CreateDocHeader(ObjDoc, "Encabezado del Documento", "Arial", 14, true, true,
                                  System.Drawing.Color.Gray,
                                  WE.Alignment.right);
            //=======================<CREACION DE LOS FOOTERS>=========================
            WEOBJ.CreateDocFooter(ObjDoc, "Página # ", "Arial", 14, true, true,
                                   System.Drawing.Color.DarkGray,
                                   WE.Alignment.right);
            //====================< CREACION DEL PARRAFO1>============================
            WEOBJ.AddTitleType(ObjDoc, "PARRAFO1", WE.HeadingType.Heading1, "Arial", 16, false, true,
                               System.Drawing.Color.Black,
                               WE.Alignment.both);
            WEOBJ.AddParagraph(ObjDoc, "Este es el texto demostrativo del primer párrafo de prueba de creación de un
documento Word mediante programa. " +
                               "Aquí pretendemos mostrar la relativa facilidad con que se crea un documento Word con este
conjunto de métodos.",
                               "Arial",
                               10, false, true, System.Drawing.Color.DarkRed, WE.Alignment.both);
            WEOBJ.AddParagraph(ObjDoc, "Este es el segundo párrafo demostrativo de este programa de ejemplo, es
conveniente aclarar que nuestro objetivo " +
                               "principal es simplificar al máximo la programación de aplicaciones orientadas al Word.",
                               "Arial",
                               10, false, false, System.Drawing.Color.DarkGreen, WE.Alignment.both);
            //-------------------------------------------------------------------------------------------------------
            var TABLA = WEOBJ.AppendTable(ObjDoc, 3, 3);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.PutHeaderName(TABLA, 0, "NOMBRE", "Arial", 14, System.Drawing.Color.Black);
            WEOBJ.PutHeaderName(TABLA, 1, "EDAD", "Times New Roman", 14, System.Drawing.Color.DarkRed);
            WEOBJ.PutHeaderName(TABLA, 2, "SEXO", "Courrier New", 14, System.Drawing.Color.DarkMagenta);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.PutInfoCell(TABLA, "HECTOR",1,0);
            WEOBJ.PutInfoCell(TABLA, "75",1,1);
            WEOBJ.PutInfoCell(TABLA, "M",1, 2);
            WEOBJ.PutInfoCell(TABLA, "IRMA",2,0);
            WEOBJ.PutInfoCell(TABLA, "60",2,1);
            WEOBJ.PutInfoCell(TABLA, "F",2, 2);
            //-------------------------------------------------------------------------------------------------------
            ObjDoc.InsertTable(TABLA);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.AddParagraph(ObjDoc, "AAA.", "Arial", 10, false,  true, System.Drawing.Color.DarkRed,
                               WE.Alignment.both);
            WEOBJ.AddParagraph(ObjDoc, "BBB.", "Arial", 10, false,  false, System.Drawing.Color.DarkRed,
                               WE.Alignment.both);
            //-------------------------------------------------------------------------------------------------------
            var TABLA1 = WEOBJ.AppendTable(ObjDoc, 4, 3);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.PutInfoCell(TABLA1, "Elvis",1,0);
            WEOBJ.PutInfoCell(TABLA1, "17",1,1);
            WEOBJ.PutInfoCell(TABLA1, "M",1, 2);
            WEOBJ.PutInfoCell(TABLA1, "Gretel",2,0);
            WEOBJ.PutInfoCell(TABLA1, "40",2,1);
            WEOBJ.PutInfoCell(TABLA1, "F",2, 2);
            WEOBJ.PutInfoCell(TABLA1, "Brandon",3,0);
            WEOBJ.PutInfoCell(TABLA1, "1",3,1);
            WEOBJ.PutInfoCell(TABLA1, "M",3, 2);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.PutHeaderNameItalic(TABLA1, 0, "NOMBRE", "Arial", 18, System.Drawing.Color.Red);
            WEOBJ.PutHeaderNameItalic(TABLA1, 1, "EDAD", "Arial", 18, System.Drawing.Color.Red);
            WEOBJ.PutHeaderNameItalic(TABLA1, 2, "SEXO", "Arial", 18, System.Drawing.Color.Red);
            ObjDoc.InsertTable(TABLA1);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.AddParagraph(ObjDoc, "XXX.", "Arial", 10, false, true, System.Drawing.Color.DarkRed,
                                WE.Alignment.both);
            WEOBJ.AddParagraph(ObjDoc, @"https://www.sunpass.com/", "Arial", 10, false,  true,
                                                 System.Drawing.Color.DarkRed, WE.Alignment.left);
            //-------------------------------------------------------------------------------------------------------
            WEOBJ.InsertImageWordDoc(ObjDoc, @"D:\PAISAJE.jpg");
            WEOBJ.AddParagraph(ObjDoc, "XXX.", "Arial", 10, false, true, System.Drawing.Color.DarkRed,
                               WE.Alignment.both);
            ObjDoc.ReplaceText("AAA.", "Secuencia de reemplazo de AAA.");
            ObjDoc.ReplaceText("BBB.", "Secuencia de reemplazo de BBB.");
            ObjDoc.ReplaceText("XXX.", "Señor XXX, de ahora en adelante se llamará YYY.");
            WEOBJ.CloseWordDoc(ObjDoc);
            OBJ.MensINFO(@"Terminación Exitosa del Programa - Resultado en D:\\AAA.docx");
            Application.Exit(); // Fin del Proceso
            }
        }
    }
```

Documento .docx Generado

A continuación se muestra el contenido del documento Word generado ("AAA.docx") en el disco D:\

PARRAFO1

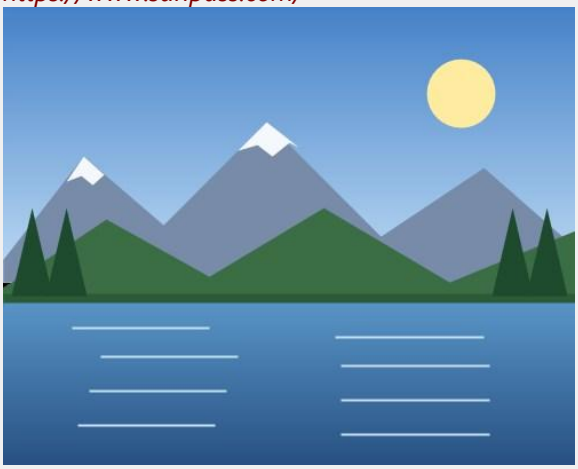
Este es el texto demostrativo del primer párrafo de prueba de creación de un documento Word mediante programa. Aquí pretendemos mostrar la relativa facilidad con que se crea un documento Word con este conjunto de métodos.
Este es el segundo párrafo demostrativo de este programa de ejemplo, es conveniente aclarar que nuestro objetivo principal es simplificar al máximo la programación de aplicaciones orientadas al Word.

NOMBRE	EDAD	SEXO
HECTOR	75	M
IRMA	60	F

Secuencia de reemplazo de AAA.
Secuencia de reemplazo de BBB.

NOMBRE	*EDAD*	*SEXO*
Elvis	17	M
Gretel	40	F
Brandon	1	M

Señor XXX, de ahora en adelante se llamará YYY.
https://www.sunpass.com/



***
Señor XXX, de ahora en adelante se llamará YYY.
Con este sencillo ejemplo se muestra la forma de trabajo de los métodos de Word de la bilioteca WE.

Métodos de Excel

Metodo ExcelCreateDocExcel() de Creación de un Documento EXCEL con su Sheet Inicial

==========

Formato:

XLWorkbook ExcelCreateDocExcel(string DocExcel, string SheetName)

Descripción:

El método ExcelCreateDocExcel() es usado solamente para crear un nuevo documento EXCEL e inmediatamente después de su ejecución se debe ejecutar el método:
XLWorkbook ExcelOpenFile(string PathFile)
Esto es obligatorio siempre y cuando se desea ejecutar acciones propias de la definición de todas las características que queramos agregar a dicho documento EXCEL, tales como los títulos de encabezamiento de cada columna, los colores de una celda (Cell), el tipo de letra de una celda, el tamaño, y así sucesivamente.
Al término de la ejecución de este método se depositan dos nombres importantes para el trabajo con EXCEL en dos variables globales accesibles por el programador:
"ExcelFile" -> Contiene el nombre interno a utilizar por los diferentes métodos y que esta asociado al documento EXCEL que se está generando.
"ExcelSheet" -> Es el nombre de la hoja (sheet) que contiene la tabla de filas y columnas con la que se va a trabajar.

Nota Importante:

Si a continuación de la ejecución del método ExcelCreateDocExcel se desean usar los diferentes métodos existentes para la creación de una hoja de cálculo determinada, entonces se debe ejecutar en forma obligatoria al método ExcelOpenFile, caso contrario se produce una excepción de programa.

Esto se debe a que al final del código de ExcelCreateDocExcel se ejecuta una instrucción workbook.SaveAs(DocExcel);

que realiza la grabación en disco del nuevo documento, cerrándolo y dejando a la hoja Excel vacia, a fín de que posteriormente sea definida bajo la utilización de los distintos métodos que daran el formato deseado.

Por tal razón usualmente se debe ejecutar a continuación el correspondiente ExcelOpenFile que abrirá las puertas al programador para poder usar cualquiera de los métodos destinados a esta finalidad.

Parámetros de entrada:
1) string DocExcel -> string que define el Path y el nombre de fichero que se desea
Ejemplo: @"D\DocumentosExcel\Alumnos.xlsx"
2) string SheetName -> string que define el nombre de la hoja Excel (sheet) que vamos a llenar con los diferentes métodos definidos con esta finalidad.

Parámetros de salida:
El nombre interno del documento Excel que se usará en varios parámetros de los métodos definidos. Este nombre es del Tipo de Dato "XLWorkbook".
==========

METODO ExcelOpenFile(string PathFile)

(Apertura de un Documento Excel)

==========

Formato:
XLWorkbook ExcelOpenFile(string PathFile)

Descripción:
El método ExcelOpenFile(string PathFile) tiene el objetivo de abrir el documento en blanco creado anteriormente por el método "ExcelCreateDocExcel". Esto es obligatorio a fin de evitar un excepción que cancelará su tarea.

Parámetros de entrada:
1) string PathFile -> string que define el Path y el nombre de fichero creado en el paso anterior.
Ejemplo: @"D\DocumentosExcel\Alumnos.xlsx"
Parámetros de salida: No tiene
Retorno:
El nombre interno del documento Excel que se usará en varios parámetros de los métodos definidos. Este nombre es del Tipo de Dato "XLWorkbook" y es requerido por varios de los métodos a utilizar en la generación del documento.
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.
==========

METODO ExcelSaveFile() DE SALVA DEL DOCUMENTO EXCEL

==========

Formato:
void ExcelSaveFile(string FileName)

Descripción:
El método "void ExcelSaveFile(string FileName)", es usado para hacer la grabación del documento generado por programa con el Path y el nombre del fichero dado en el operando string FileName.
Nota Importante:
Este método finaliza el trabajo con el documento que se genera, por lo que si se desea dentro del mismo programa continuar trabajando, es obligatorio volver a usar el método "ExcelOpenFile", pues en caso contrario se produce una excepción.

Parámetros de entrada:
1) string FileName -> string que define el Path y el nombre de fichero creado en el paso anterior.
Ejemplo: @"D\DocumentosExcel\Alumnos.xlsx"
Parámetros de salida: No tiene

Retorno:
No tiene (void)

==========

METODO ExcelFormatTitles() DE ESCRITURA DE UN TITULO COLUMNAR (Row = 1, Column = n)

==========

Formato:

```
void ExcelFormatTitles(int ColumnCell,
                       string ValueCell,
                       string FontName,
                       int FontSize,
                       bool FontBold,
                       bool FontItalic,
                       XLColor BackColor, XLColor ForeColor)
```

Descripción:

Los títulos de cada columna definidos por el usuario en EXCEL pueden ser variados y con un formato de estilo definido por el programador. Dado que dichos títulos columnares deben aparecer en la línea 1 (row = 1) pues no es requerido como parámetro y por tal motivo el primero que aparece es un entero que define la columna del título que se desea.

Parámetros de entrada:
ColumnCell - Número de la columna.
ValueCell - Es un string que define el título de la columna.
FontName - Es un string que define la letra que se va a usar ("Arial", "Courrier New", etc.
FontSize - El tamaño de la letra (12, 16, etc.)
FontBold - Variable de tipo bool que si esta en true es que se quiere letra negrita.
FontItalic - Variable de tipo bool que si esta en true es que se quiere letra Itálica.
BackColor - Variable de la clase XLColor que define el color de Background (fondo)
ForeColor - Variable de la clase XLColor que define el color de Foreground (letra)
Parámetros de salida: No tiene
Retorno: No tiene (void)
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ExcelFormatCell() DE DAR FORMATO A UNA CELDA

==========

Formato:

```
void ExcelFormatCell(int RowCell,
                     int ColumnCell,
                     string ValueCell,
                     string FontName,
                     int FontSize,
                     bool FontBold,
                     bool FontItalic,
                     XLColor BackColor, XLColor ForeColor
                     )
```

Descripción:

El método "ExcelFormatCell" es usado para asignar un valor a una celda bajo el formato de (fila, columna) con estilo de color, tipo de letra, tamaño de letra, etc.
Parámetros de entrada:
RowCell - Número de fila.
ColumnCell - Número de columna.
ValueCell - Es un string que define el valor de la columna.
FontName - Es un string que define la letra que se va a usar ("Arial", "Courrier New", etc.
FontSize - El tamaño de la letra (12, 16, etc.)
FontBold - Variable de tipo bool que si esta en true es que se quiere letra negrita.
FontItalic - Variable de tipo bool que si esta en true es que se quiere letra Itálica.
BackColor - Variable de la clase XLColor que define el color de Background de la celda
ForeColor - Variable de la clase XLColor que define el color de Foreground de la celda
Parámetros de salida: No tiene
Retorno: No tiene (void)
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ExcelWriteCellRC() DE ESCRITURA DE UNA CELDA DEL

DOCUMENTO EXCEL

==========

Formato:

```
void ExcelWriteCellRC(int RowCell,
                      int ColumnCell,
                      string ValueCell)
```

Descripción:

El método "ExcelWriteCellRC" es usado para asignar un valor a una celda bajo el formato de (fila, columna).
Parámetros de entrada:
RowCell - Número de fila.
ColumnCell - Número de columna.
ValueCell - Es un string que define el valor de la columna.
Parámetros de salida: No tiene
Retorno: No tiene (void)
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ExcelWriteCell() DE ESCRITURA DE UNA CELDA DEL DOCUMENTO EXCEL

==========

Formato:

```
void ExcelWriteCell(string InputCell,
                    string ValueCell)
```

Descripción:

El método "ExcelWriteCell" es usado para asignar un valor a una celda bajo el formato de (string).
Parámetros de entrada:
InputCell - Celda de salida en formato de string, ejemplo "B2", "C5", etc.
ValueCell - Es un string que define el valor que tendrá la columna.
Parámetros de salida: No tiene
Retorno: No tiene (void)
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ExcelGetCellRC() DE ESCRITURA DE UNA CELDA CON NOMBRE EN string

==========

Formato:

void ExcelGetCellRC(int RowCell,
int ColumnCell)

Descripción:

El método "ExcelGetCellRC" es usado para obtener el valor a una celda bajo el formato (R, C)
Parámetros de entrada:
RowCell - Número de fila de la Celda de entrada.
ColumnCell - Número de columna de la Celda de entrada.
Parámetros de salida: No tiene
Retorno: No tiene (void)
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ""ExcelGetCell" DE LECTURA DE UNA CELDA DEL DOCUMENTO EXCEL

==========

Formato:
string ExcelGetCell(string InputCell)

Descripción:

El método "ExcelGetCell" es usado para obtener el valor a una celda bajo el formato string de Excel.
Parámetros de entrada:
InputCell - string que referencia a la Celda de entrada, ej, "A4" , "B2", etc.
Parámetros de salida: No tiene
Retorno: Devuelve el valor de la celda en formato string.
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

==========

METODO ExcelGetCellDataTypeRC() DE LECTURA DEL TIPO DE DATOS DE UNA CELDA (r, c)

==========

Formato:

string ExcelGetCellDataTypeRC(int RowCell,
int ColumnCell)

Descripción:

El método "ExcelGetCellDataTypeRC" es usado para obtener el tipo de datos de una celda bajo el formato (r, c) de Excel.
Parámetros de entrada:
RowCell - int que da el valor de la fila de la Celda de entrada.
ColumnCell - int que da el valor de la columna de la Celda de entrada.
Parámetros de salida: No tiene
Retorno: Devuelve el tipo de datos de la celda de Excel en formato string.
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ExcelGetCellDataType() DE LECTURA DEL TIPO DE DATO DE UNA CELDA EN FORMATO "string"

==========

Formato:

string ExcelGetCellDataType(string InputCell)

Descripción:
El método "ExcelGetCellDataType" es usado para obtener el tipo de datos de una celda bajo el formato string de Excel.
Parámetros de entrada:
InputCell - Dirección de la celda de entrada en formato string de EXCEL ("A4", "B6", etc.)
Parámetros de salida: No tiene
Retorno: Devuelve el tipo de datos de la celda de Excel en formato string.
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.

==========

METODO ExcelFormula() PARA DESCRIBIR UNA FORMULA EN EXCEL

==========

Formato:
void ExcelFormula(int CellDef, string Formula)

Descripción:

Método para definir una formula Excel.
Parámetros de entrada:
ValueCell - Es un string que define el título de la columna.
Formula - Es un string que define la letra que se va a usar ("Arial", "Courrier New", etc.

Parámetros de salida: No tiene
Retorno: No tiene (void)
Si se detecta algún error en la apertura se emite el mensaje correspondiente con la excepción detectada.
==========

Programa de ejemplo de generación de un documento EXCEL

```
using ClosedXML.Excel;
using DocumentFormat.OpenXml.Vml;
using System;
using System.Windows.Forms;
namespace PruebaExcelNuget
   {
   //=================================================================
   // Autor: Lic. Hector Fortun (22/Enero/2024)
   //=================================================================
   // Programa de ejemplo de creación y modificación de documentos
   // EXCEL bajo el uso de los métodos adicionados a la biblioteca
   // WE  creada por mi para trabajar con conceptos
   // que usualmente consumen bastante tiempo de programación.
   // Esta biblioteca abarca varios perfiles de trabajo de diferente
   // tipo como son por ejemplo, métodos de validación de la información,
   // creación y modificación de documentos MS Word o su similar
   // Ashampoo Writer, métodos para mensajería, trabajo con fecha y hora,
   // llamadas a procedimientos y objetos de Windows, Mensajes de Error
   // o Informativos, etc. Todo la anterior, siempre con el objetivo
   // principal de hacer fácil lo difícil y ahorrar tiempo.
   //
   // El objetivo principal ha sido el de crear una lógica simple y a la
   // vez suficiente para facilitarle al programador de C# bajo
   // Visual Studio el trabajo con documentos Excel, no solamente del
   // MSOffice, sino también de otros productos similares como el Office
   // de Ashampoo que acepta las hojas de cálculo .xlsx y xls.
   //
   // Nota Importante:
   //
   // Dado que originalmente el poder programar para MSExcel requería
   // que estuviera instalado el MSOffice en su PC por usar a una librería
   // requerida por Microsoft (Interop), pues decidimos apoyarnos en
   // algunos Package de Nuget, como DocumentFormat.OpenXml (para Word) y
   // ClosedXML (para EXCEL), ambos con licencia MIT.
   // Dada mi experiencia personal con los anteriores observé que se
   // resistian un poco a romper la inercia y decidí estructurar a estas
   // herramientas de una forma muy simple  con nuevos métodos que
   // ayudaran al programador a avanzar con rapidez en la puesta a punto
   //  rápida de las tareas mas cotideanas.
   // No obstante, hemos dejado dejado las puertas abiertas para que
   // cuando el programador lo decida pues utilice las herramientas
   // originales del fabricante, al cual le estamos muy agradecidos por
   // tener la idea de poder trabajar con equivalentes al Word y al Excel
   // de Microsoft sin necesidad de tenerlos instalados en nuestra PC.
   // Es conveniente aclarar que ambos paquetes permiten el uso libre, también
   // en aplicaciones de índole comercial.

   //=================================================================
```

```
public partial class Form1 : Form
      {
      //---------------------------------------------------------
      // Variables Globales Para Excel
      //---------------------------------------------------------
      XLWorkbook      ExcelFile   = null;
      IXLWorksheet    ExcelSheet  = null;
      //---------------------------------------------------------
      // A continuación se instancia al objeto denominado
      // OBJ con la clase WE de la biblioteca multipropósito
      // WE. Por tal razón se hará referencia a este objeto
      // frecuentamente dentro del programa.
      //---------------------------------------------------------
      WE OBJ = new WE();
      //---------------------------------------------------------
      int          C = 0;
      int          F = 0;
      string       B2 = "";
      string       B3 = "";
      string       B4 = "";
      string       celda = "";
      float        B2Float = 0;
      float        B3Float = 0;
      float        B4Float = 0;
      float        B6Float = 0;
      string      TipoDato = "";
      XLDataType  XX = new XLDataType();
      //---------------------------------------------------------------
      public Form1()
          {
          InitializeComponent();
          }

      private void button1_Click(object sender, System.EventArgs e)
          {
          try
              {
              // Apertura del nuevo documento Excel que se va a generar
              // el cual a su vez va a contener al primer sheet "Hoja1".

              OBJ.ExcelCreateDocExcel(@"D:\AAANuevo.xlsx", "Hoja1");

              // Abrir al fichero creado en el paso anterior para
              // trabajar con el.

              OBJ.ExcelFile  = OBJ.ExcelOpenFile(@"D:\AAANuevo.xlsx");

              // Abrir la hoja de cálculo (sheet) y asignarla a
              // ExcelSheet
              OBJ.ExcelSheet = OBJ.ExcelOpenSheet(1);

              OBJ.ExcelFormatTitles(1, "NOMBRE", "Segoe UI", 18, true, true,
                                    XLColor.Red, XLColor.Yellow);
              OBJ.ExcelFormatTitles(2, "SALARIO", "Segoe UI", 18, true, true,
                                    XLColor.Red, XLColor.Yellow);
              OBJ.ExcelFormatCell(2, 1, "Arial", "Arial", 18, true, false,
                                    XLColor.DarkBlue, XLColor.Turquoise);
              OBJ.ExcelFormatCell(3, 1, "Times New Roman", "Times New Roman",
                                    16, true, false,
                                    XLColor.Black, XLColor.White);
              OBJ.ExcelFormatCell(4, 1, "Courrier New", "Courrier New",
                                    14, true, false,
                                    XLColor.Red, XLColor.Yellow);
              OBJ.ExcelFormatCell(2, 2, "250.50", "Times New Roman",
                                    18, true, false,
                                    XLColor.Turquoise, XLColor.DarkBlue);
```

```
OBJ.ExcelFormatCell(3, 2, "300.48", "Arial", 16, true, false,
                        XLColor.White, XLColor.Black);
OBJ.ExcelFormatCell(4, 2, "600.44", "Segoe UI", 14, true, false,
                        XLColor.Yellow, XLColor.Red);

OBJ.ExcelSheet.Row(1).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(2).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(3).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(4).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(5).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(6).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

//=========OPERACIONES ARITMETICAS CON CELDAS EN UNA COLUMNA=======
B2 = OBJ.ExcelSheet.Cell("B2").Value.ToString();
B3 = OBJ.ExcelSheet.Cell("B3").Value.ToString();
B4 = OBJ.ExcelSheet.Cell("B4").Value.ToString();
//---------------------------------------------------------------------
B2Float = float.Parse(B2);
B3Float = float.Parse(B3);
B4Float = float.Parse(B4);
//---------------------------------------------------------------------
B6Float = B2Float + B3Float + B4Float;
//---------------------------------------------------------------------
OBJ.ExcelSheet.Cell("B6").Value = B6Float.ToString();

//------------FORMATEAR LA COLUMNA DEL RESULTADO--------------------
OBJ.ExcelFormatCell(6, 2, OBJ.ExcelSheet.Cell("B6").Value.ToString(),
                    "Arial", 18, true, false,
                    XLColor.DarkRed, XLColor.Yellow);
//---------------------------------------------------------------------
// El resultado de esta operacion aritmética "(B4 + B6 + A7) / 3"
// queda en A8
//---------------------------------------------------------------------
// Es conveniente observar que la celda B5 no tiene contenido,
// sin embargo se acepta en la operación como si el valor fuera cero.
//---------------------------------------------------------------------
OBJ.ExcelFormula("A8", "(B4 + B5 + B6 + A7) / 3");

//============ASIGNACION DE UN TIPO DE DATOS A UNA CELDA============
OBJ.ExcelSheet.Cell(7,1).Value = "123.45";
B2 = OBJ.ExcelSheet.Cell(7,1).DataType.ToString();
XX = OBJ.ExcelSheet.Cell(7,1).DataType;
OBJ.ExcelFormatCell(7, 1, OBJ.ExcelSheet.Cell(7,1).Value.ToString(),
                    "Arial", 14, true, false,
                    XLColor.DarkRed, XLColor.Yellow);
celda = OBJ.ExcelGetCell("A7");
XX = XLDataType.Number;

if (XX == XLDataType.Text)
```

```
                {
                OBJ.MensINFO("Cell A7 = Text");
                }
            if (XX == XLDataType.Number)
                {
                OBJ.MensINFO("Cell A7 = Number");
                }
            //---------------------------------------------------------------------
            OBJ.ExcelWriteCellRC(5, 1, "AAA");
            OBJ.ExcelWriteCellRC(6, 1, "BBB");

            OBJ.ExcelWriteCell("B3", "CCC");
            OBJ.ExcelWriteCell("B2", "DDD");
            // Obtiene el valor de la celda (3,2)
            celda = OBJ.ExcelGetCellRC(3, 2);
            //Obtiene el tipo de Dato de (4,2)
            TipoDato = OBJ.ExcelGetCellDataTypeRC(4, 2);
            //Obtiene el tipo de Dato de "B2"
            celda = OBJ.ExcelGetCellDataType("B2");

            OBJ.MensINFO("# Columnas: " + C.ToString() + "\n" +
                         "# Filas: " + F.ToString() +
                         "\nCelda(3,2) = " + celda +
                         "\nTipo de Dato (4,2) = " + TipoDato +
                         "\nTipo de Dato (2,2) = " + celda);

            OBJ.MensINFO("Fin del Proceso");
            }
        catch
            {
            OBJ.MensINFO("Excepción Detectada - Se cancela el programa\n" +
                         e.ToString());
            Application.Exit();
            }
        OBJ.ExcelSaveFile(@"D:\AAAOutput.xlsx");
        Application.Exit();
        }
    }
}
```

==========

METODOS DE DataGridView

==========

Método DGVColumnSumarize()

==========

FORMATO:

```
int DGVColumnSumarize(DataGridView DGV,
                      int DGVColumn,
                      ref double SUMARIA)
```

DESCRIPCION:

El método DGVColumnAverageSelectedRows() realiza el cálculo de la suma de las cantidades que se encuentran en las celdas de una columna dada por DGVColumn del DataGridView del primer operando, dejando el resultado en la variable de tipo double del tercer operando SUMARIA, por referencia.

Parámetros de entrada:

DataGridView DGV
int DGVColumn
ref double SUMARIA

Parámteros de salida:

ref double SUMARIA -> Transmisión por referencia de la suma columnar

Retorno:

Un valor entero que informa como fué la terminación de la ejecución del método.

Ejemplo:

```
DGVColumnSumarize(DataGridView1, 4, ref SumaTotal);
```

==========

Método DGVColumnSumarizeSelectedRows()

==========

FORMATO:

```
int DGVColumnSumarizeSelectedRows(DataGridView DGV,
                                  int DGVColumn,
                                  ref double SUMARIA)
```

DESCRIPCION:

El método DGVColumnAverageSelectedRows() realiza el cálculo de la suma de las cantidades que se encuentran en las celdas seleccionadas de una columna dada por DGVColumn del DataGridView del primer operando, dejando el resultado en la variable de tipo double del tercer operando SUMARIA, por referencia.

Parámetros de entrada:

DataGridView DGV
int DGVColumn
ref double SUMARIA

Parámteros de salida:

ref double SUMARIA -> Transmisión por referencia de la suma columnar

Retorno:

Un valor entero que informa como fué la terminación de la ejecución del método.

Ejemplo:

```
DGVColumnSumarizeSelectedRows(DataGridView1, 4, ref SumaTotal);
```

==========

==========

Método DGVColumnAverage()

==========

FORMATO:

```
public int DGVColumnAverage(DataGridView DGV,
                            int DGVColumn,
                            ref double AVERAGE)
```

DESCRIPCION:

El método DGVColumnAverage() realiza el cálculo del promedio de las cantidades que se encuentran en las celdas de una columna dada por DGVColumn del DataGridView del primer operando, dejando el resultado en la variable de tipo double del tercer operando AVERAGE, por referencia.

Parámetros de entrada:

DataGridView DGV
int DGVColumn
ref double AVERAGE

Parámteros de salida:

ref double AVERAGE -> Transmisión por referencia

Retorno:

Un valor entero que informa como fué la terminación de la ejecución del método.

Ejemplo:

```
DGVColumnAverage(DataGridView1, 4, ref Promedio);
```

==========

Método DGVColumnAverageSelectedRows()

==========

FORMATO:

```
int DGVColumnAverageSelectedRows(DataGridView DGV,
                                 int DGVColumn,
                                 ref double AVERAGE)
```

DESCRIPCION:

El método DGVColumnAverageSelectedRows() realiza el cálculo del promedio de las cantidades que se encuentran en las celdas seleccionadas de una columna dada por DGVColumn del DataGridView del primer operando, dejando el resultado en la variable de tipo double del tercer operando AVERAGE, por referencia.

Parámetros de entrada:

DataGridView DGV
int DGVColumn
ref double AVERAGE

Parámteros de salida:

ref double AVERAGE -> Transmisión por referencia

Retorno:

Un valor entero que informa como fué la terminación de la ejecución del método.

Ejemplo:

```
DGVColumnAverageSelectedRows(DataGridView1, 4, ref Promedio);
```

==========

==========

Método DGVGetCell()

==========

FORMATO:

```
string DGVGetCell(DataGridView DGV,
                  int DGVRow,
                  int DGVColumn)
```

DESCRIPCION:

El método DGVGetCell() retorna el contenido de la celda (DGVRow, DGVColumn) en formato de string

Parámetros de entrada:

DataGridView DGV
int DGVRow
int DGVColumn

Parámteros de salida:

No tiene

Retorno:

Retorna el contenido de la celda (DGVRow, DGVColumn) en formato de string.

Ejemplo:

```
DGVGetCell(DataGridView1, 4, 1);
```

Retorna el contenido la celda (4,1) con formato de string.

==========

Método DGVPutCell()

==========

FORMATO:

```
void DGVPutCell(DataGridView DGV,
                string CellValue,
                int DGVRow,
                int DGVColumn)
```

DESCRIPCION:

El método DGVPutCell() escribe el contenido de CellValue en la celda (DGVRow, DGVColumn) del DataGridView en formato de string.

Parámetros de entrada:

DataGridView DGV
string CellValue
int DGVRow
int DGVColumn

Parámteros de salida:

No tiene

Retorno:

No tiene.

Ejemplo:

```
DGVPutCell(DataGridView1, "Irma", 4, 1);
```

Escribe a la cadena "Irma" en la celda (4,1) en formato de string.

==========